Implementation of Retrieval-Augmented Generation (RAG) with the Qwen 2.5 7B Instruct Generator in a Stunting Question-Answering System

Muhammad Andrata[1], Irfan Maliki[2]

[1,2] Indonesian Computer University
Jalan Dipatiukur No. 112-116, Coblong, Kota Bandung, Jawa Barat 40132.
E-mail : andrata2908@gmail.com[1], irfan.maliki@email.unikom.ac.id[2]

Abstract

Stunting is a chronic nutritional problem that remains a serious public health issue in Indonesia, with a national prevalence of 19.8% in 2024. Although various intervention programs have been implemented, public knowledge about stunting remains low, reflected in only 1.7% of national respondents having good knowledge and attitudes. Large Language Models (LLMs) offer an opportunity as interactive question-answering systems to bridge this information gap, but their implementation is still limited by hallucination and knowledge cut-off. Retrieval-Augmented Generation (RAG) is proposed as a solution by integrating the retrieval of factual information from external sources before the model generates an answer. This study implements a RAG pipeline using a Qwen 2.5 7B Instruct generator quantized with NF4 4-bit, supported by a knowledge base of 60 official documents indexed into 12,558 chunks through FAISS. Testing on the Gold Standard Dataset shows strong retrieval performance with an average MRR of 0.9469 and Context Precision of 0.9565, along with generation performance averaging 0.8988 Faithfulness and 0.9223 Answer Relevancy. These results demonstrate that RAG significantly reduces hallucination compared to No-RAG, which failed completely on numerical questions with 0% relevance, although hallucination challenges remain for numerical data, with Faithfulness dropping to as low as 68.75%. The application of the RAG architecture proves capable of optimizing small-scale language models to produce credible, relevant answers grounded in factual evidence for stunting education in Indonesia.

Keywords : Retrieval-Augmented Generation, Qwen 2.5 7B, Stunting, LLM Hallucination, Question Answering System.

1. INTRODUCTION

Stunting is a condition of impaired growth and development in children caused by chronic undernutrition and recurrent infections, and it remains a serious public health problem in Indonesia [1]. besides hindering physical growth and triggering metabolic disorders in adulthood, stunting lowers children's cognitive ability and causes national economic losses of up to 2–3% of Gross Domestic Product (GDP) every year [1]. The 2024 Indonesian Nutritional Status Survey (SSGI) recorded a national stunting prevalence of 19.8%, down from 21.6% in 2022, yet still above the government target of 14% [2]. The same data show that the causes of stunting are multidimensional, encompassing maternal nutrition, recurrent infections, the quality of complementary feeding, access to sanitation, and the educational and economic conditions of families [2].

Government efforts to reduce this figure through the interventions regulated in Presidential Regulation No. 72 of 2021 are still constrained by the limited number and capacity of *posyandu* cadres and by uneven program coverage [1]. As a result, public knowledge of stunting remains low, with SSGI 2024 recording that only 1.7% of respondents nationwide have good knowledge of and attitudes toward stunting [2], consistent with the finding that 50.3% of adolescent girls have never been exposed to information on stunting from adequate sources [3]. This gap underscores the need for self-service educational channels that can reach the public without relying entirely on cadre capacity.

Advances in Natural Language Processing (NLP) and Large Language Models (LLMs) open opportunities to build interactive question answering systems that can bridge this information gap [4], [5], with a more human-like conversational interface that the public can access directly [5], [6]. However, the use of LLMs in the health domain is still limited by *hallucination* and *knowledge cut-off* [7], [8], [9], LLMs can produce answers that sound convincing but are inaccurate because they rely on statistical patterns of language rather than verified knowledge [10], [11], a serious risk in medical domains such as stunting because it can affect health safety and decision making [7], [12].

Retrieval-Augmented Generation (RAG) addresses these limitations by integrating information retrieval from external factual sources before the LLM generates a response [13], [14], [15], thereby reducing *hallucination* and ensuring that answers remain grounded in valid data [13], [16]. This approach has proven effective in the health domain. RAG on medical knowledge comprehension tasks raised the average evaluation score by up to 3.86% compared with the base model [15], while integrating knowledge graphs into RAG increased the ROUGE-1 score by up to 9.72% on *multi-hop question answering* tasks [14]. Beyond the health domain, RAG has also been shown to preserve contextual relevance and accuracy, for example in LLM-based assessment systems in higher education [17], which indicates its potential for application in health domains such as stunting. These findings support the recommendation that LLMs in the health domain use appropriate corpora as knowledge bases, in line with prevailing regulations and practices [10].

Despite this considerable potential [4], [14], [15], research on RAG for the Indonesian-language stunting domain remains scarce, because most RAG research still centers on Western-language models. The representation of Indonesian in LLaMA-2, for instance, is only 0.03% of the total *pre-training* data, with a performance gap of up to 20% in *weighted F1-score* compared with specifically optimized models [18], [19]. To fill this gap, this study uses the Qwen 2.5 7B Instruct model, chosen for its accessibility and *deployment* efficiency, since its lower computational requirements make it realistic to run on affordable hardware at regional health institutions [20], [21], without sacrificing language capability because Qwen 2.5 is proficient in around 30 languages including Indonesian [20], [22] and scored 92.2 in the *LLM-as-a-Judge* evaluation for Indonesian-language RAG tasks [23].

The objective of this study is to implement a RAG *pipeline* based on Indonesian-language stunting documents, from corpus preparation and vector representation building to integration with the Qwen 2.5 7B Instruct model as the answer generation engine, and to analyze its performance empirically through a structured evaluation framework, in order to demonstrate the extent to which RAG augmentation improves the response quality of small-scale models as a foundation for developing similar systems in the future [4], [24].

2. METHODOLOGY

This study applies a quantitative experimental approach organized into six stages of system design: 1) Collection and preparation of the stunting *knowledge base*; 2) Design of the stunting corpus preparation; 3) Design of the RAG architecture; 4) Generation of the *Gold Standard* evaluation dataset; 5) Design of the evaluation system; and 6) Design of the Gradio-based QA prototype interface. This *end-to-end* workflow uses the LangChain orchestration *library* in the Google Colaboratory environment with NVIDIA Tesla T4 GPU acceleration and 16 GB of VRAM. This *end-to-end* workflow is illustrated in Figure 1.

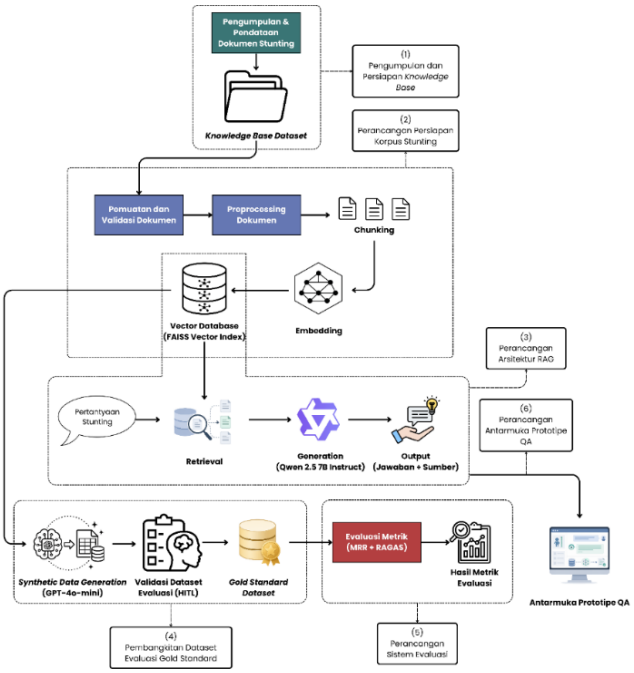


Figure 1. End-to-End Stunting RAG Diagram

2.1 Knowledge Base Collection and Preparation

The *knowledge base* is built selectively by collecting a corpus of stunting-related documents published between 2014 and 2026 from credible official institutional sources, such as the Ministry of Health of the Republic of Indonesia (Kemenkes), the National Population and Family Planning Board (BKKBN), the Ministry of Villages, Development of Disadvantaged Regions, and Transmigration (Kemendes PDTT), and the Secretariat of the Vice President (Setwapres), as well as several local governments and accredited scientific journal repositories. This curation process yielded 60 heterogeneous documents, grouped into five credibility tiers according to document type based on regulatory and legal authority in Indonesia. The distribution of document types in the *knowledge base* is presented in detail in Table 1.

Table 1. Knowledge base distribution by document type

Document Type	Count	Document Coverage
National Regulations	11	National regulations such as presidential regulations, ministerial regulations on health, and laws
Technical Guidelines	19	Official technical guidelines from ministries and agencies
Government Data	16	Quantitative or qualitative data or strategy documents
Regional Regulations	6	Regional regulations such as governor regulations, regent regulations, and regional head decrees
Scientific Journals	8	Verified scientific journals indexed in Sinta or Scopus
Total	**60**	**-**

All *knowledge base* documents are classified and stored in a centralized directory and registered in the metadata file verified_documents_metadata.json, which acts as the *single source of truth*. Of the 60 documents, 48 are original PDF files from the publishers, 8 are Markdown (.md) files converted with the marker library from PDFs dominated by complex tables, and 4 are DOCX files converted from quantitative data in Excel spreadsheets into structured narrative sentences.

2.2 Stunting Corpus Preparation Design

After the documents are verified, the corpus flows through a *data pipeline* consisting of four sequential stages. The first stage is document loading, designed to be *format-aware*. Digital PDF pages are extracted using PyMuPDF, scanned pages are handled through Tesseract OCR as a *fallback* path, Word documents are processed through a DOCX XML parser that iterates directly over the body elements of the document so that the original content order is preserved, and Markdown documents are handled by a *custom block parser*. The second stage is text *preprocessing*, designed to remove semantic noise. It includes PDF-specific cleaning, consisting of the removal of institutional URL watermarks through *regex* and the normalization of physical line breaks produced by PyMuPDF extraction, and universal cleaning, consisting of the elimination of residual HTML tags from conversion, the normalization of ligature characters, and the insertion of spaces between numbers and nutritional units, such as converting the Indonesian string "6bulan" into "6 bulan". At this stage, non-substantive structural content such as the Table of Contents and References is also filtered out using a combination of *dotted leader*, trailing digit, and *keyword anchor* signals, so that these structural documents do not compete in the retrieval results.

The third stage is *chunking*, which splits documents into text segments ready for embedding through two paths according to the characteristics of their content, as summarized in Table 2.

Table 2. Chunking strategy configuration

Content Path	Method	Chunk Size	Overlap
Narrative Text	RecursiveCharacterTextSplitter	1,000 characters	200 characters
Tabular Data	Row-based Semantic Splitting	1,200 characters	2 rows

On the tabular path, table rows are serially converted into structured declarative sentences in the format "[Context: Table Name] Column1: Value | Column2: Value" to preserve the semantic relations among medical data. After the documents are split through their respective paths, all *chunks* are collected and pass through seven sequential filtering and optimization stages, namely noise removal of empty *chunks* or syntax

artifacts, sweeping of residual structural components that escaped the initial *preprocessing* layer, context injection in the form of inserting the parent sentence into *chunks* containing list items so that they do not lose meaning when retrieved separately, merging of short *chunks* from the same page to maintain information density, filtering of *chunks* that remain below the minimum length threshold after merging, deduplication that retains only *chunks* from documents with the highest credibility tier, and metadata injection in the form of inserting the document title and source at the start of the *chunk* text. The strict application of metadata at this stage has proven important for retrieval precision in question answering systems [25].

The fourth stage is *embedding* and indexing. Each final *chunk* is given the prefix "passage: " before being converted into a vector representation using the intfloat/multilingual-e5-base model [26], following the input format required by E5 models to distinguish document representations from user *query* representations. The model produces 768-dimensional vectors with a normalized vector norm of $\|v\| = 1$, so that Euclidean distance on the IndexFlatL2 index structure of the FAISS library is mathematically equivalent to cosine distance. All vectors, together with the metadata of their source documents, are stored in a FAISS *vector store*, which becomes the indexed stunting corpus used in the next stage, the design of the RAG architecture.

2.3 RAG Architecture Design

After the corpus preparation stage produces a FAISS index ready to serve as the source of answers, the next stage is to design the RAG architecture that serves user questions in *real time*. This architecture is the inference layer of the system, working by integrating two main components, namely the retrieval component and the generation component, as illustrated in Figure 2.

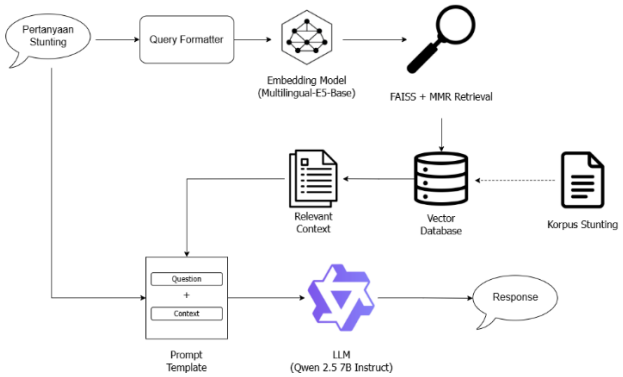


Figure 2. Overview of the Stunting RAG Pipeline.

In general, the *pipeline* workflow begins when a user asks a question about stunting. The question is first formatted by a *query formatter* with the prefix "query: " before being converted into a vector representation by the same multilingual-e5-base *embedding* model used during indexing, because the model requires different input formats for document text and question text so that both lie in a comparable representation space. This vector representation is then used to perform a semantic search on the FAISS index to obtain a number of relevant document segment candidates, which are subsequently selected using the Maximal Marginal Relevance (MMR) algorithm to balance relevance to the question with diversity among the retrieved documents, thereby reducing the redundancy of the context sent to the LLM compared with a *pure similarity search* approach [27]. The document segments resulting from MMR still carry the "passage: " prefix from the indexing stage and are therefore cleaned before being passed to the generation stage, with two retrieval paths tailored to their needs, namely the chatbot interface path that returns only clean content without additional metadata, and the evaluation path that includes complete metadata along with similarity scores for the computation of MRR and RAGAS metrics in the testing stage.

In the generation component, the user question is combined with the retrieved context into a structured *prompt template* following the ChatML format according to Qwen model conventions. The template consists of a *system message* with six instruction blocks, namely the identity of StuntBot, the stunting domain context, a strict prohibition on mentioning technical metadata or using *self-referential* phrases such as "based on the document above", a temporal prohibition on ambiguous words such as "currently" or "now", a rule to refuse questions outside the stunting domain, and a rule requiring the opening sentence to answer the core of the question directly. The composed prompt is then passed to the Qwen 2.5 7B Instruct model to generate the final response, which runs using the Normal Float 4-bit (NF4) quantization technique

through the bitsandbytes library to reduce the VRAM memory load on the Tesla T4 GPU. All parameter configurations for both components are summarized in Table 3. Experimental results demonstrate that NF4 4-bit quantization reduced the model size by up to 64.3% and lowered peak VRAM requirements by up to 56.0% without sacrificing the substantive quality of the answers, thereby leaving room for the *embedding* component and the retrieval process to run concurrently on the 16 GB T4 GPU.

Table 3. Parameter configuration of the retrieval and generation components

Component	Parameter	Value
Retrieval	k	5
Retrieval	fetch_k	30
Retrieval	lambda_mult	0.85
Generation	temperature	0.1
Generation	max_new_tokens	1,024
Generation	top_p	0.95
Generation	repetition_penalty	1.05

2.4 Gold Standard Evaluation Dataset Generation

To measure system performance objectively, the Gold Standard Dataset was built automatically using synthetic data generation (SDG) with GPT-4o-mini as the generator through the OpenRouter API, following a *generate-then-filter* paradigm. Candidate question-answer-context triples were generated from 2,714 unique *chunks* that had passed pre-generation filters, and were then validated through three automated layers, namely JSON schema validation, a *groundedness check* to detect answer hallucination, and *Round-Trip Consistency* to ensure that the source *chunk* could be retrieved again by the *retriever* within the top 5. The process was halted once 80 QA pairs had been collected, distributed across six question types, namely 24 factual, 18 causal, 14 procedural, 10 multi-context, 8 numerical, and 6 policy.

This automatically generated dataset was then passed to a manual validation *pipeline* based on *Human-in-the-Loop* (HITL) to maintain accuracy and accountability [17], involving two evaluators. The researcher, designated Evaluator 1, measured Source Groundedness between the answers and the source *chunks* in the FAISS index, while a nutrition expert, designated Evaluator 2, validated medical correctness and linguistic clarity on three dimensions, namely Faithfulness, Answer Consistency, and Clarity, on a scale of 1 to 3. QA pairs scoring 1 on Faithfulness or Answer Consistency were rejected and regenerated, a score of 2 on either dimension led to revision, and a score of 3 on all dimensions led to full acceptance. This iteration produced 80 final high-quality QA pairs as the sole reference for system evaluation.

2.5 Perancangan Sistem Evaluasi

The evaluation system assesses the *pipeline* from two aspects. The retrieval component is evaluated using MRR to assess the ranking of relevant documents, along with two RAGAS metrics, namely Context Precision, which measures how free the retrieval results are from irrelevant *chunks*, and Context Recall, which measures the completeness of information relative to the *ground truth*. The generation component is evaluated using three RAGAS metrics, namely Faithfulness, which measures the factual consistency of the answer with the context, Answer Relevancy, which measures how precisely the answer addresses the question, and Answer Correctness, which measures the accuracy of the answer against the expert *ground truth*. The computation of RAGAS metrics uses GPT-4o-mini as the *judge model* [28]. In addition, a comparative evaluation between the full RAG scenario and a No-RAG scenario was conducted on the same 80 questions to measure the contribution of retrieval in reducing hallucination.

2.6 QA Prototype Interface Design and Realization

As a validation of *pipeline* functionality in a real user interaction scenario, the system is realized as a single-page web application using the Gradio framework. The interface is divided into a question input column with example questions on the left, and an answer output column on the right equipped with an *expandable panel* of reference documents that displays the title, source, year, page number, and similarity score of each retrieved document, as a form of transparency about answer sources to users. The initial view of the prototype interface is shown in Figure 3.



Figure 3. Initial view of the QA prototype interface

3. RESULTS AND DISCUSSION

3.1 Retrieval Component Performance Analysis

The performance of the retrieval component was measured to confirm how effectively the system finds relevant supporting documents from the FAISS database containing 12,558 *chunks*. Testing on 80 questions produced an average MRR of approximately 0.95, a Context Precision of approximately 0.96, and a Context Recall of approximately 0.87. A detailed breakdown of performance by question type is presented in Table 4.

Table 4. Retrieval component performance breakdown by question type

Question Type	Count (n)	MRR (Mean)	Context Precision	Context Recall
Factual	24	0.9375	0.9608	0.8757
Causal	18	0.9444	0.9836	0.9213
Procedural	14	0.9107	1.0000	0.9524
Multi-context	10	1.0000	1.0000	0.8250
Numerical	8	0.9375	0.7505	0.7500
Policy	6	1.0000	0.9593	0.7778
Overall Aggregate	**80**	**0.9469**	**0.9565**	**0.8731**

The system recorded a retrieval *hit rate* of 100%, meaning that not a single question failed to find a relevant document within the top 5 ranking. Most questions (90%) successfully placed the relevant document exactly at rank 1. The multi-context and policy types achieved perfect MRR scores, confirming that policy documents and cross-context sources have highly distinctive vector representations in the FAISS index, making them very easy for the *embedding* model to recognize.

On the other hand, the numerical question type recorded the lowest performance, with Context Precision and Context Recall both falling in the range of 0.75. This decline is attributable to the characteristics of the tabular stunting data, such as tables of under-five prevalence figures per province, which, after passing through *row-based semantic splitting*, produced many short *chunks* with near-identical numerical content. The MMR algorithm identified the high semantic similarity among these numerical *chunks* as redundancy, thereby excluding some important numerical data *chunks* from the top-5 results list and causing the loss of part of the context needed to answer numerical questions comprehensively

3.2 Generation Component Performance Analysis

The performance of the generation component measures the quality of the Qwen 2.5 7B Instruct model's responses when augmented with retrieval context. The aggregate test results recorded an average Faithfulness of approximately 0.90, an Answer Relevancy of approximately 0.92, and an Answer

Correctness of approximately 0.73. The distribution of generation metrics by question type is shown in Table 5.

Table 5. Generation component performance breakdown by question type

Question Type	Count (n)	Faithfulness	Answer Relevancy	Answer Correctness
Factual	24	0.9493	0.9120	0.7340
Causal	18	0.9114	0.9292	0.7798
Procedural	14	0.8751	0.9362	0.7424
Multi-context	10	0.8964	0.9067	0.7120
Numerical	8	0.6875	0.9442	0.6646
Policy	6	1.0000	0.9075	0.6466
Overall Aggregate	**80**	**0.8988**	**0.9223**	**0.7295**

Answer Relevancy is the metric with the most consistent and tightly clustered distribution among the three, indicating that the system was almost always able to answer directly on point without producing rambling information that strayed off-topic. The policy type recorded perfect Faithfulness, because the formal, normative, and assertive style of regulatory language pushed the model to quote directly and literally without adding opinions or inferences beyond the text.

Conversely, the numerical type emerged as the main critical weak point in the generation component, with the lowest average Faithfulness at around 0.69. Two numerical questions recorded 0% Faithfulness, meaning that every numerical claim in the model's response was either incorrect or absent from the context. This condition correlates strongly with the low Context Recall value for the numerical type. When the *retriever* failed to fully present the numerical table data segments due to MMR *pruning*, the small-parameter language model tended to fill the resulting information gap by hallucinating numbers from its internal parametric memory. The analysis results demonstrate that, in the stunting context, RAG reliably reduces hallucination in narrative-conceptual content but remains vulnerable to failure on numerical, table-based content.

3.3 Comparative Evaluation of RAG vs. No-RAG Performance

To demonstrate the real contribution of the retrieval component augmentation, a comparative test was conducted by feeding the same 80 questions directly to the Qwen 2.5 7B Instruct model without the aid of the retrieval mechanism. The comparison was limited to the Answer Relevancy (AR) and Answer Correctness (AC) metrics, because neither requires context input in its computation. The comparison results by question type are presented in Table 6.

Table 6. Comparison of generation metric performance between the RAG and No-RAG scenarios

Type	AR RAG	AR No-RAG	AR Difference	AC RAG	AC No-RAG	AC Difference
Factual	0.9120	0.8084	+0.1036	0.7340	0.5917	+0.1423
Causal	0.9292	0.8597	+0.0695	0.7798	0.6383	+0.1415
Procedural	0.9362	0.8433	+0.0929	0.7424	0.4444	+0.2980
Multi-context	0.9067	0.9035	+0.0032	0.7120	0.6416	+0.0704
Numerical	0.9442	0.0000	+0.9442	0.6646	0.2210	+0.4436
Policy	0.9075	0.4527	+0.4548	0.6466	0.4481	+0.1985
Aggregate	**0.9223**	**0.7304**	**+0.1919**	**0.7295**	**0.5348**	**+0.1947**

Based on Table 6, the application of RAG yields a consistent and significant improvement in average performance, with Answer Relevancy increasing by approximately 19% and Answer Correctness also increasing by approximately 19%. Three of the most prominent patterns can be identified from this comparison. First, for the numerical type, the No-RAG scenario recorded 0% Answer Relevancy across all test questions, demonstrating that the general-purpose language model has no parametric memory capacity for local stunting prevalence statistics and Indonesian health program percentages, resulting in outright refusal or total failure without RAG. With RAG, relevancy jumped sharply to above 90%. Second, for the policy type, No-RAG recorded an AR of only around 45%, because regional regulations such as the East Nusa Tenggara Governor Regulation or the West Papua Governor Regulation are highly domain-specific and were never encountered by the model during its general *pre-training* phase. RAG corrected this limitation by raising AR to around 91%.

A third pattern is evident in the procedural type, where the Answer Relevancy gap between RAG and No-RAG is only about 9%, yet the Answer Correctness gap reaches nearly 30%. Qualitative analysis of the No-RAG model's answers shows that the model without retrieval was able to construct procedural answer sentences that sounded highly convincing, coherent, and structured as medical steps. However, when checked against the Ministry of Health's official standard operating procedures, the substance of those nutritional steps proved incorrect and misleading. This finding underscores the danger of hallucination hidden behind a convincing linguistic style that goes undetected from linguistic relevance alone, and demonstrates the vital role of RAG in aligning model answers with the health policies currently in force in Indonesia.

Overall, the test results show that the constructed RAG architecture, comprising a 60-document *knowledge base*, a FAISS index of 12,558 *chunks*, MMR retrieval, and integration with Qwen 2.5 7B Instruct, performs as designed and delivers strong performance across all six evaluation metrics, with Answer Correctness as the only metric that is structurally lower, due to differences in elaboration style relative to the expert reference answers. The overall performance profile across the six metrics is shown in Figure 4.

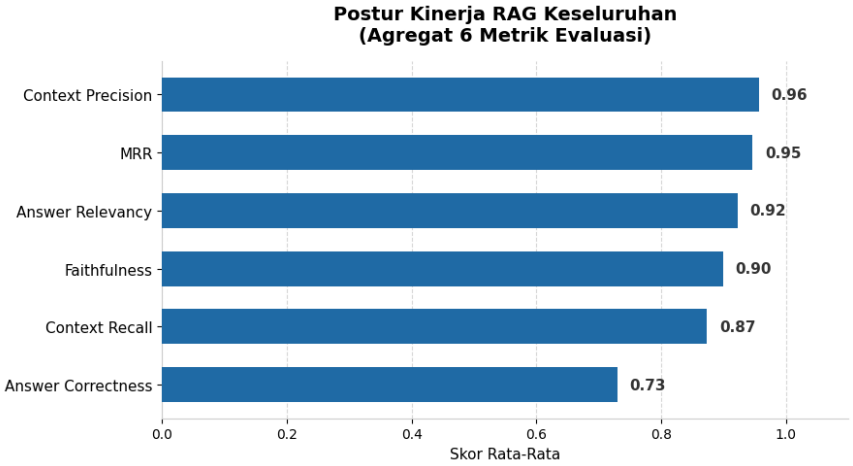


Figure 4. Overall RAG performance profile (aggregate of 6 evaluation metrics)

4. CONCLUSION

The implementation of RAG with the Qwen 2.5 7B Instruct generator for the Indonesian-language stunting question answering domain has been successfully realized through an integrated pipeline, from the preparation of a 60-document knowledge base to a FAISS index containing 12,558 chunks, with NF4 4-bit quantization proven to run efficiently on a T4 GPU without sacrificing the model's ability to follow instructions in Indonesian. Testing using the Gold Standard Dataset showed strong performance across all evaluation metrics, with MRR reaching approximately 0.95 and average Faithfulness of approximately 0.90, confirming that RAG significantly reduces hallucination compared with the No-RAG scenario. Even so, the system still showed weaknesses for the numerical question type, with Faithfulness dropping to around 0.69 due to the characteristics of tabular data that are difficult for the current chunking and retrieval strategy to handle. This shortcoming opens avenues for further research, including exploring vision-language model-based table parsing and hybrid search mechanisms to improve recall on numerical data and cross-document questions, periodically expanding and updating the knowledge base given the time-sensitive nature of health documents, conducting direct testing with the public and healthcare workers beyond synthetic data, applying ablation testing and assessing system robustness to input variation, and

further developing the system into a more scalable production application to broaden the reach of stunting health education for the Indonesian public.

ACKNOWLEDGMENTS

The author would like to thank everyone who helped bring this research to completion, especially the thesis supervisor and the nutrition expert for their guidance and rigor in keeping this research accurate. Thanks to family for their unwavering support, and to someone who was always present throughout every step of the process.

REFERENCES

[1] T. Wahyuni, R. A, D. Sartika, M. Kusumaningrum, and M. A. Sari, "Implementasi Kebijakan Pemerintah Dalam Penurunan Prevalensi Stunting," *J. Ris. Inossa Media Has. Ris. Pemerintahan, Ekon. dan Sumber Daya Alam*, vol. 6, no. 02, pp. 38–50, 2025, doi: 10.54902/jri.v6i02.161.

[2] Kemenkes RI, "Survei Status Gizi Indonesia 2024 Dalam Angka," 2025. [Online]. Available: https://www.badankebijakan.kemkes.go.id/survei-status-gizi-indonesia-ssgi-2024/

[3] N. Nurhayati, N. Kurwiyah, R. Rohanah, S. D. Paramita, and A. D. Putri Atifa, "Keterpaparan informasi dan tingkat pengetahuan tentang stunting pada remaja putri," *Holistik J. Kesehat.*, vol. 17, no. 8, pp. 688–696, 2023, doi: 10.33024/hjk.v17i8.12937.

[4] G. Samudra, A. T. Zy, and Ermanto, "Implementasi Retrieval Augmented Generation (RAG) Dalam Perancangan Chatbot Kesehatan Pencernaan," *JSAI J. Sci. Appl. Informatics*, vol. 8, no. 1, pp. 1–8, 2025.

[5] D. Sarkar, "Navigating the Knowledge Sea: Planet-scale answer retrieval using LLMs," *arXiv*, 2024, [Online]. Available: https://arxiv.org/abs/2402.05318

[6] S. H. Kumar *et al.*, "Cue-bot: A Conversational Agent for Assistive Technology," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2022, pp. 196–203. doi: 10.18653/v1/2022.acl-demo.19.

[7] L. K. Umapathi, M. Sankarasubbu, and A. Pal, "Med-HALT: Medical Domain Hallucination Test for Large Language Models," *27th Conf. Comput. Nat. Lang. Learn. Proc.*, pp. 314–334, 2023, doi: https://dx.doi.org/10.18653/v1/2023.conll-1.21.

[8] V. Agarwal, Y. Jin, M. Chandra, M. De Choudhury, S. Kumar, and N. Sastry, "MEDHALU : Hallucinations in Responses to Healthcare Queries by Large Language Models," *Proc. Twent. Int. AAAI Conf. Web Soc. Media*, vol. 20(1), no. Icwsm, pp. 31–44, 2026.

[9] M. Omar *et al.*, "Multi-model assurance analysis showing large language models are highly vulnerable to adversarial hallucination attacks during clinical decision support," *Commun. Med.*, vol. 5, no. 1, pp. 1–7, 2025, doi: 10.1038/s43856-025-01021-3.

[10] Z. Li *et al.*, "Evaluating clinical competencies of large language models with a general practice benchmark," *Nat. Commun.*, pp. 1–12, 2026.

[11] C. Garcia Fernandez, "Trustworthy AI for medicine: Continuous hallucination detection and elimination with CHECK," *Ann. Oncol.*, vol. 36, p. S1654, 2025, doi: 10.1016/j.annonc.2025.08.457.

[12] S. Anjum, H. Zhang, W. Zhou, E. J. Paek, X. Zhao, and Y. Feng, "HALO: Hallucination Analysis and Learning Optimization to Empower LLMs with Retrieval-Augmented Context for Guided Clinical Decision Making," in *Proceedings - 2025 IEEE/ACM International Conference on Connected Health: Applications, Systems and Engineering Technologies, CHASE 2025*, 2024, pp. 187–198. doi: 10.1145/3721201.3721385.

[13] A. Boulesnane and A. Souilah, "An Evolutionary Large Language Model for Hallucination Mitigation," in *Proceedings - ECTE-Tech 2024: International Conference on Electrical, Computer, Telecommunication, and Energy Technologies*, 2024. doi: 10.1109/ECTE-TECH62477.2024.10851107.

[14] Q. Wei, H. Ning, C. Han, and J. Ding, "A query-aware multi-path knowledge graph fusion approach for enhancing retrieval-augmented insgeneration in large language models," *Expert Syst. Appl.*, vol. 316, p. 131932, 2026, doi: https://doi.org/10.1016/j.eswa.2026.131932.

[15] Y. Wang, Y. Wan, X. Lei, Q. Chen, and H. Hu, "A retrieval augmented generation based optimization approach for medical knowledge understanding and reasoning in large language models," *Array*, vol. 28, no. September, p. 100504, 2025, doi: 10.1016/j.array.2025.100504.

[16] D. K. Pham and B. Q. Vo, "Towards Reliable Medical Question Answering: Techniques and

Challenges in Mitigating Hallucinations in Language Models," 2024, [Online]. Available: http://arxiv.org/abs/2408.13808

[17] R. Adha, D. Syaripudin, B. K. S, A. M. Bachtiar, H. Maulana, and E. Rainarli, "Integrasi Gemini AI Berbasis Retrieval-Augmented Generation (RAG) pada Moodle untuk Penilaian Esai Otomatis dengan Pendekatan Human-in-the-Loop di Pendidikan Tinggi," *Acad. J. Comput. Sci. Res.*, vol. 8, no. 1, pp. 67–74, 2026.

[18] S. Cahyawijaya *et al.*, "Cendol: Open Instruction-tuned Generative Large Language Models for Indonesian Languages," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 14899–14914. doi: 10.18653/v1/2024.acl-long.796.

[19] D. S. adillah Maylawati, Y. J. Kumar, F. B. Kasmin, and M. A. Ramdhani, "Readability Evaluation Metrics for Indonesian Automatic Text Summarization: A Systematic Review," 2024. doi: 10.25103/jestr.175.25.

[20] Qwen *et al.*, "Qwen2.5 Technical Report," 2025, [Online]. Available: https://arxiv.org/abs/2412.15115

[21] W. Christian, D. Adamlu, A. Yu, and D. Suhartono, "Bridging Language Gaps with Adaptive RAG: Improving Indonesian Language Question Answering," 2025, [Online]. Available: http://arxiv.org/abs/2510.21068

[22] A. Yang *et al.*, "Qwen2 Technical Report," pp. 1–26, 2024, [Online]. Available: https://arxiv.org/abs/2407.10671

[23] H. Lijaya, P. Ho, and H. Santoso, "Comparative Analysis of RAG-Based Open-Source LLMs for Indonesian Banking Customer Service Optimization Using Simulated Data," *J. Sisfokom (Sistem Inf. dan Komputer)*, vol. 14, no. 3, pp. 330–341, 2025, doi: 10.32736/sisfokom.v14i3.2383.

[24] D. Firdaus, I. Sumardi, and Y. Kulsum, "Integrating Retrieval-Augmented Generation with Large Language Model Mistral 7b for Indonesian Medical Herb," *JISKA (Jurnal Inform. Sunan Kalijaga)*, vol. 9, no. 3, pp. 230–243, 2024, doi: 10.14421/jiska.2024.9.3.230-243.

[25] K. S. Handoyo, M. L. Klau, R. J. Wijaya, and R. P. Kristianto, "Penerapan Retrieval-Augmented Generation untuk Pembuatan Soal Pilihan Ganda dari Materi Berbasis PDF," *Pros. Semin. Nas. KONSTELASI*, vol. 3, no. 1, pp. 200–209, 2026, doi: 10.24002/prosidingkonstelasi.v3i1.13516.

[26] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual E5 Text Embeddings: A Technical Report," 2024, [Online]. Available: http://arxiv.org/abs/2402.05672

[27] Y. Choi, S. Kim, Y. Cedric, and F. Bassole, "Enhanced Retrieval-Augmented Generation Using Low-Rank Adaptation," *Appl. Sci.*, vol. 15, pp. 1–19, 2025.

[28] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in *EACL 2024 - 18th Conference of the European Chapter of the Association for Computational Linguistics, Proceedings of System Demonstrations*, 2024, pp. 150–158. doi: 10.18653/v1/2024.eacl-demo.16.